

Topic 8.2

8.2: Error Handling

Error handling is difficult in all programming languages.

Characterizing the problem, we generally have the main program invoking a function A, which invokes a function B, etc. Somewhere deep in the call chain, say function Q, something goes wrong, and the programmer may desire to simply "get back" to some earlier function in the call chain, possibly the main program.

A difficulty with this in languages which utilize a runtime stack is the necessity to clean up the stack as we return from each function in the call chain. This cleanup is handled by the run-time support library and is beyond the programmer's control. Generally, this requires quite ugly code, such as the following:

```
int error_number=0;    // Global variable.

int main(void)
{
    a();
    if (error_number)
    {
        // Process error.
    }

    return 0;
}

void a(void)
{
    b();
    if (error_number) return;
}

void b(void)
{
    c();
    if (error_number) return;
}

...
void q()
{
    // Something goes wrong here.
    error_number = 13;
    return;
}
```

The code to handle the error clutters up the program, significantly reducing the readability and increasing the maintenance costs.

In C, there are a pair of routines, *setjmp* and *longjmp*, which clean this up significantly. The function *setjmp* takes a snapshot of what the runtime stack looks like at a given point during program execution. At some later time, such as when an error occurs, *longjmp* is invoked with this snapshot. It

causes the stack to be "cleaned off" so it matches the snapshot. Many snapshots can be taken to establish different levels of error handling capability.

setjmp returns a 0 value when it is first invoked to take the snapshot. At the time *longjmp* is invoked, it appears to the code that *setjmp* has been invoked again, but this time the return value will be non-zero (based on the value passed to *longjmp*), so the program can distinguish the taking of the snapshot from a rollback-to-the-snapshot.

The following C program illustrates this.

```
/*
 * PROGRAM:    err1.c
 * PROGRAMMER: Gary J. Blair
 * LANGUAGE:   ANSI C
 * DATE:      96/07/05
 *
 * PURPOSE:
 *   Illustrate use of setjmp/longjmp
 *   for error handling.
 *
 * NOTES:
 *   setjmp/longjmp are not compatible
 *   with most C++ implementations.
 */

#include <stdio>
#include <stdlib.h>
#include <setjmp.h>

jmp_buf main_prog_environment; /* Records info
                                about runtime
                                stack. */

void a(void);
void b(void);
void c(void);

int main(void)
{
    int setjmp_result;

    setjmp_result = setjmp(main_prog_environment);

    if (setjmp_result == 0)
    {
        printf("Beginning main program.\n");
        a();
        exit(0);
    }
    else /* longjmp brings us here. */
    {
        printf("*** longjmp invoked with error
              value:%d.\n", setjmp_result);
        exit(1);
    }
    return 0;
}

void a(void)
{
    b();
}
```

```

void b(void)
{
    c();
}

void c(void)
{
    longjmp(main_prog_environment,73);
}

```

Output of program err1.c:

```

Beginning main program.
*** longjmp invoked with error value:73.

```

The following program written in pure C illustrates a more realistic use of setjmp/longjmp for error handling. The program is persistent in obtaining valid inputs from the user, and specifically identifies problems with the inputs.

```

/*
 * PROGRAM:      mlgerr.c
 * PROGRAMMER:   Gary J. Blair
 * LANGUAGE:     ANSI C
 * DATE:        97/05/12
 *
 * PURPOSE:
 *   Calculate gas mileage.
 *   Illustrate use of setjmp/longjmp
 *   for error handling.
 *
 * NOTES:
 *   setjmp/longjmp are not compatible with
 *   most C++ implementations.
 */

#include <stdio.h>
#include <stdlib.h>
#include <setjmp.h>

#define INIT_ODOM_ERR    1
#define FINAL_ODOM_ERR  2
#define GAS_ERR          3

jmp_buf main_loop;    /* Records info about runtime stack. */

void read_inputs( float *init_odom,
                  float *final_odom,
                  float *gas );
float mileage( float init_odom,
               float final_odom,
               float gas );
void print_result(float mpg);

int main(void)
{
    int setjmp_result;    // Actual invocation
                        // of setjmp()
                        // or error return.
    int success = 0;      // Computation
                        // successful?

```

```

float initial_odometer; // Initial odometer
// reading in miles.
float final_odometer;   // Final odometer
// reading in miles.
float gas_used;          // Gallons of gas
// used.
float mpg;               // Miles per gallon.

while (! success)
{
    read_inputs
    (&initial_odometer,&final_odometer,&gas_used);
    setjmp_result = setjmp(main_loop);
    if (setjmp_result == 0)
    {
        mpg = mileage( initial_odometer,
        final_odometer, gas_used);
        print_result(mpg);
        success = 1;
    }
    else if (setjmp_result == INIT_ODOM_ERR)
        printf
        ("*** Error in initial odometer reading.\n");
    else if (setjmp_result == FINAL_ODOM_ERR)
        printf
        ("*** Error in final odometer reading.\n");
    else if (setjmp_result == GAS_ERR)
        printf("*** Error in amount of gas used.\n");
    }
    return 0;
}

void read_inputs( float *init_odom,
float *final_odom,
float *gas )
{
    printf("Enter initial odometer reading ");
    scanf("%f",init_odom);
    printf("Enter final odometer reading   ");
    scanf("%f",final_odom);
    printf("Enter gallons of gas used      ");
    scanf("%f",gas);
}

float mileage( float init_odom, float final_odom,
float gas )
{
    if (init_odom < 0)
        longjmp(main_loop,INIT_ODOM_ERR);
    if (final_odom < init_odom)
        longjmp(main_loop,FINAL_ODOM_ERR);
    if (gas <= 0)
        longjmp(main_loop,GAS_ERR);
    return (final_odom - init_odom) / gas;
}

void print_result(float mpg)
{
    printf("Your gas mileage is %.2f\n",mpg);
}

```

Output of program err1.c:

```
Enter initial odometer reading -19
Enter final odometer reading 23
Enter gallons of gas used 8
*** Error in initial odometer reading.
Enter initial odometer reading 230
Enter final odometer reading 200
Enter gallons of gas used 8
*** Error in final odometer reading.
Enter initial odometer reading 230
Enter final odometer reading 260
Enter gallons of gas used 3
Your gas mileage is 10.00
```

Topic 8.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-8-dot-3>)